
Algoritmos de Busca e Análise de Complexidade Assintótica

Relatório Técnico Acadêmico | ADS 2026

16 de março de 2026

Busca Linear / Busca Binária / Complexidade $O(\log N)$ / Big-O

Este documento apresenta uma síntese técnica da Unidade 2, Seção 1 da disciplina Linguagem de Programação, baseando-se nos estudos de Vanessa Cadan Scheffer.

1 Objetivos

Este relatório objetiva explorar os mecanismos de recuperação de dados através de algoritmos de busca linear e binária. A análise foca na eficiência de cada método frente ao volume de dados, utilizando a notação Big-O para quantificar o desempenho e a escalabilidade das soluções propostas no desenvolvimento de software.

2 Desenvolvimento Teórico

Conforme SCHEFFER (2020), a escolha de um algoritmo de busca é ditada pela estrutura e organização dos dados. Enquanto a simplicidade da busca sequencial atende a listas desordenadas, a busca binária é imperativa para sistemas que exigem alta performance em bases de dados extensas.

- **Busca Linear (ou Sequencial):** O algoritmo percorre a estrutura do início ao fim. Sua complexidade é $O(N)$, o que significa que o esforço cresce proporcionalmente ao número de elementos.
- **Busca Binária:** Aplica a estratégia de divisão sucessiva. Reduz o espaço de busca pela metade a cada iteração, resultando em uma complexidade $O(\log N)$. Contudo, exige que os dados estejam previamente **ordenados**.

Conceito-Chave: A eficiência algorítmica é o diferencial entre um sistema funcional e um sistema escalável. A transição de $O(N)$ para $O(\log N)$ pode reduzir milhões de operações para apenas algumas dezenas.

3 Aplicação Prática/Código

Os testes práticos realizados no Google Colaboratory demonstraram a superioridade da busca binária. Em uma lista simulada com 1,5 milhão de registros, a busca linear exigiria, no pior caso, o mesmo número de comparações, enquanto a busca binária resolveu o problema com aproximadamente 21 operações.

```
# Implementação de Busca Binária ( $O \log N$ )
def busca_binaria(lista, alvo):
    baixo = 0
    alto = len(lista) - 1

    while baixo <= alto:
        meio = (baixo + alto) // 2
        chute = lista[meio]
        if chute == alvo:
            return f"Elemento encontrado na posição {meio}!"
        if chute > alvo:
            alto = meio - 1
        else:
            baixo = meio + 1
    return "Elemento não localizado."
```

A prática validou a teoria de SCHEFFER (2020): a organização prévia dos dados (ordenação) é um investimento computacional que compensa drasticamente a velocidade de recuperação da informação.

Conclusão

A análise da Unidade 2.1 evidencia que o domínio da complexidade assintótica é essencial para o profissional de ADS. A compreensão da notação Big-O permite projetar arquiteturas otimizadas, garantindo que a experiência do usuário não seja degradada pelo crescimento da base de dados, mantendo a agilidade e a viabilidade técnica do produto final.

Referências

1. SCHEFFER, Vanessa Cadan. **Linguagem de Programação**. Unidade 2, Seção 1. Livro Digital. Londrina: Editora e Distribuidora Educacional S.A., 2020.
2. GUIMARÃES, Murilo. **Exercícios Práticos e Simulações**. Google Colaboratory, 2026. Disponível em: <https://colab.research.google.com/drive/1FKh69bNB711EF240anrbwYdW2qWqrmlp?usp=sharing>.

